Project 4 — ContentForge-Research-Write-Review-Team-Turns-Any-Topic-Into-A-Publish-Ready-Article

Title: ContentForge Research Write Review Team Turns Any Topic Into A Publish Ready Article

Type: Multi-agent (5 agents: 1 Supervisor + 4 specialists) · **Stack:** Python, LangGraph, `Command` routing, LangChain · **Level:** Upper-Intermediate

Tagline: A supervisor-led team — Research, Analysis, Writer, and Reviewer — that takes a topic from looked-up facts to a reviewed, revised final article.

> This is the **reading guide** — the story, the folder structure and why each part exists, setup, the 5 build steps, and which document to read for each idea. The other file, README.md, is the **repo README** — you copy it into your repo folder at the end and push it to GitHub. You write every folder, file and line of code yourself; the only thing already created for you is the empty repo folder `ContentForge-Research-Write-Review-Team-Turns-Any-Topic-Into-A-Publish-Ready-Article/` .

Charter (what this project is)

A Supervisor routes four specialists — Research → Analysis → Writer → Reviewer — through one shared LangGraph state, using `Command`, with a hard-limited revision loop between Writer and Reviewer. It's the headline project of the curriculum: **one agent vs. many**, proven with a system you can defend. It's also built the way real multi-agent systems are built: one specialist, proven alone, at a time — never all five at once.

Jump to: Read first · Project structure · Setup · How testing works · Step 1 · Step 2 · Step 3 · Step 4 · Step 5 · Checklist · Publish

The Story — what you're actually building

You give a coworker one topic — "electric bikes" — and later they hand back a short, sourced, reviewed article, ready to publish. They didn't do it alone. Someone looked the facts up, someone organized them into themes, someone wrote the draft, someone read it critically and sent it back until it was good enough — and a manager kept the whole team on track.

That's ContentForge. A **Supervisor** (the manager) reads the shared state and decides who works next, routing with LangGraph's `Command`. Four specialists each have one narrow job and a prompt written only for that job: **Research** looks facts up with a tool, **Analysis** groups them into themes without losing any, the **Writer** drafts from the findings only, and the **Reviewer** scores the draft against written criteria and sends it back with feedback — up to a hard limit.

The build order matters as much as the design. Step 1 proves the Writer alone. Step 2 pairs it with a Reviewer in a plain loop. Steps 3 and 4 add Research and Analysis, each proven alone and then with the real output of the one before. Only Step 5 brings in the Supervisor and the graph — when every specialist is already trusted.

What you're building, in one line: a Supervisor that routes four specialists through a shared state with `Command`, turning a topic into a reviewed article.

Why this needs to exist: one prompt that researches, organizes, writes and critiques at once blurs all four jobs — and a wrong fact early on flows straight into the final text with nothing to catch it.

When you'd reach for this at a real job: a task with clearly separate phases needing different skills — a content pipeline (research → draft → review → publish), a multi-step analysis tool.

How it works, mechanically: every specialist node ends with `Command(goto="supervisor")`. The Supervisor looks at the state, picks the next specialist with a written rule, logs the reason, and returns `Command(goto=...)`. The Reviewer clears a rejected draft, which sends the work back to the Writer — until it passes or `MAX_REVISIONS` is reached.

Why not one "write me an article" call? Nothing separates checking facts from writing prose, so early mistakes go straight through. **Why not plain sequential code?** You'd still hand-write the revision loop, the shared state and the routing — a graph with a Supervisor makes them explicit, logged and testable.

Where This Fits in the 5-Project Arc

```
Project 1               → 2 agents, Python router, raw SDK
Project 2               → 2 agents, hand-built agent loop, tools
Project 3               → 3 agents, LangGraph, RAG + human approval
Project 4 (you are here) → 5 agents, one Supervisor directs the rest
                          (Supervisor + Research/Analysis/Writer/Reviewer)
Project 5               → Project 4, made production-ready
```

New here: **dynamic routing**. Project 3's graph had fixed conditional edges; here a Supervisor decides the route at run time with `Command`.

What to Read First (docs this project uses)

You need to understand…	Read this	Used in
`prompt | model | parser`	Doc05 — LCEL basics	Steps 1-4
`with_structured_output(Model)`	Doc05 — structured parser	Step 2
`@tool`, a tool tested alone	Doc06 — first tool	Step 3
Reading a JSON file	Doc02 — JSON	Step 3
A loop with a hard limit	Doc07 — step limit	Step 2
One agent or many?	Doc11 — the real question	Charter
Generator → critic, supervisor	Doc11 — patterns	Steps 2, 5
`Command(update=..., goto=...)`	Doc11 — the Command tool	Step 5
Shared vs. private state	Doc11 — shared state	Step 5
Logging which agent did what	Doc11 — logging	Step 5
A loop that won't converge	Doc11 — convergence exercise	Steps 2, 5
The whole team, as a practice run	Doc11 Build Task	Step 5

Reused code, unchanged except for its folder and import lines: `config.py`, `exceptions.py`, `utils/logger.py` (Doc01), `utils/prompts.py` (Project 1), `models/llm.py` (Project 3). The Supervisor, the `routing_log` with reasons, the feedback history and the "clear the draft to revise" rule are Doc11's Build Task (Intermediate, Approach 2).

Project Structure (the professional layout)

Same rule as before: only folders with a real job. New here: an `agents/` folder with five agents, and a `tools/` folder for Research's lookup.

Names: the repo folder vs. the Python package

- **Repo folder:** `ContentForge-Research-Write-Review-Team-Turns-Any-Topic-Into-A-Publish-Ready-Article/` — already created, and the exact GitHub repo name.
- **Python package:** `src/`, right next to `main.py`. It holds all the app's code, and every file imports from it the same way: `from src.graph import build_graph`.
- **Files:** `snake_case.py`, one job each; agent files end in `_agent.py` (the Supervisor is `supervisor.py`); test files start with `test_`.
- **Prompt files:** named after their agent (`writer.txt` → `writer_agent.py`).

The final layout (after Step 5)

```
<repo folder>/   (ContentForge-Research-...-Publish-Ready-Article/)
├── data/
│   └── sample/
│       └── facts.json          the facts Research can look up
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── analysis_agent.py   run_analysis(), analysis_node
│   │   ├── research_agent.py   run_research(), research_node
│   │   ├── reviewer_agent.py   run_reviewer(), reviewer_node
│   │   ├── supervisor.py       decide_next(), supervisor()
│   │   └── writer_agent.py     run_writer(), writer_node
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py              get_model()             (Project 3)
│   ├── prompts/
│   │   ├── analysis.txt
│   │   ├── research.txt
│   │   ├── reviewer.txt
│   │   └── writer.txt
│   ├── tools/
│   │   ├── __init__.py
│   │   └── lookup.py           lookup_facts (@tool)
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── logger.py           get_logger()            (Doc01)
│   │   └── prompts.py          load_prompt(name)       (Project 1)
│   ├── __init__.py
│   ├── config.py               Config, load_config()   (Doc01)
│   ├── exceptions.py           MissingConfigError      (Doc01)
│   ├── graph.py                build_graph(): 5 nodes, 1 edge
│   ├── schemas.py              ReviewVerdict (score, feedback)
│   └── state.py                TeamState, start_state()
├── tests/
│   ├── integration/
│   │   └── test_live_agents.py real API calls
│   └── unit/
│       ├── test_config.py      no API key needed
│       ├── test_lookup.py      no API key needed
│       └── test_supervisor.py  no API key needed
├── .env                        your real key — never committed
├── .env.example                the setting names, no real values
├── .gitignore                  keeps .env, .venv/, __pycache__/ out
├── LICENSE                     lets others reuse your code (MIT)
├── main.py                     asks for a topic, prints the report
```

```
├── README.md                    the repo's front page (copied in at end)
└── requirements.txt             the libraries, for pip install -r
```

Why each folder exists

- **`data/sample/`** — the fact store Research reads. **Why:** facts with sources you know, so you can check the article didn't invent any.
- **`src/`** — the package, right next to `main.py`: every other `.py` file the app is made of. `state.py`, `graph.py` and `schemas.py` sit at its top level — they describe the whole team, not one agent.
- **`agents/`** — one file per agent. Each file holds the agent's plain `run_...()` function (its skill, testable alone) and, from Step 5, a small node wrapper for the graph. **Why:** five jobs, five files — "why 4 agents and not 1" is something you can point at.
- **`models/`** — builds the chat model once for all five agents.
- **`prompts/`** — one prompt per specialist. **Why:** each agent's narrow job *is* its prompt; keeping them apart and readable is the whole point of splitting the work.
- **`tools/`** — the Research agent's lookup tool. **Why:** a tool is tested on its own before an agent uses it, so a wrong note can be pinned on the tool or the agent.
- **`utils/`** — the logger and the prompt loader.
- **`tests/integration/`** — the agents and the whole team against the real API.
- **`tests/unit/`** — the config, the lookup tool and the Supervisor's routing rules, with hand-made inputs. **Why:** routing bugs are found for free, without running the team.

Why each root file exists

- `.env` — your real API key; never committed.
- `.env.example` — which settings to create, with no real values.
- `.gitignore` — keeps secrets, the venv and cache files out of git.
- `LICENSE` — the MIT license text, created in Setup. Without one, nobody may legally reuse your code.
- `main.py` — the file you run: it reads a topic, runs the team and prints the report.
- `README.md` — the repo's front page on GitHub.
- `requirements.txt` — the library list for `pip install -r requirements.txt`.

Why each file exists (what and why)

Every file you'll write, in one place — the same What/Why list the documents give after each tree. The steps below build these files one at a time; come back here whenever you forget why a file exists.

- `data/sample/facts.json`

    - **What:** A JSON object mapping each topic to a list of short facts, each ending with a `[source]` tag.
    - **Why:** You know these facts, so an invented fact in the article is easy to spot.
- `agents/analysis_agent.py`
    - **What:** `run_analysis(notes)` runs the analysis prompt chain and returns the themed findings. Plus `analysis_node(state)`, which returns `Command(update={findings}, goto="supervisor")`.
    - **Why:** The Writer gets a clear structure, and the "keep every fact" rule means nothing is lost on the way.
- `agents/research_agent.py`
    - **What:** `run_research(topic)` calls `lookup_facts.invoke({"topic": topic})`; if it returns `""`, it logs a warning and returns `""`; otherwise it runs the research prompt chain and returns the notes. Plus `research_node(state)`, which calls it and returns `Command(update={notes, research_done: True}, goto="supervisor")`.
    - **Why:** The article starts from looked-up facts, not the model's memory, and an unknown topic stops the team instead of producing invented facts.
- `agents/reviewer_agent.py`
    - **What:** `PASS_SCORE`, `run_reviewer(findings, draft)` (a chain with `with_structured_output(ReviewVerdict)`), and `reviewer_node(state)`: on a pass, set `accepted`; on a fail, add 1 to `revision_count`, add the feedback to the history, and clear the draft if revisions are left.
    - **Why:** A second opinion with written criteria stops weak drafts. Clearing the draft is the signal that sends the work back to the Writer.
- `agents/supervisor.py`
    - **What:** `MAX_REVISIONS`, `decide_next(state)` — a chain of `if` checks in a fixed order that returns `(next_step, reason)` — and `supervisor(state)`, which adds `{step, reason}` to the routing log and returns `Command(goto=...)` (or `goto=END`).
    - **Why:** The routing rules are a plain function you can test with a hand-made dict, and every move is logged with its reason.
- `agents/writer_agent.py`
    - **What:** `build_writer_chain()`, `run_writer(findings, feedback="")` (turns empty feedback into "none" and invokes the chain), and `writer_node(state)`, which passes the newest feedback from `feedback_history` and returns the draft.
    - **Why:** It writes the article, and it can improve the draft when the Reviewer asks — using the Reviewer's exact feedback.
- `models/llm.py`
    - **What:** A `MODEL_NAME` constant and one function, `get_model()`, which returns `ChatOpenAI(model=MODEL_NAME, temperature=0)`.
    - **Why:** Every chain and agent gets the same model and settings from here. To try another model you change one line, not every file.

- `prompts/*.txt`
  - **What:** Four text files, one per specialist: research (one note per fact, keep the tag), analysis (themes, keep every fact), writer (only the findings, with a feedback section), reviewer (three written criteria and the scoring rule).
  - **Why:** Each agent's narrow job is written down in its own prompt — the reason for splitting the work in the first place.
- `tools/lookup.py`
  - **What:** `load_facts()` reads `facts.json` with `json.loads()`, and a `@tool` function `lookup_facts(topic)` returns the facts of the first known topic found in the text, one per line — or `""`.
  - **Why:** Research's tool can be tested without any model, so a wrong note can be pinned on the tool or on the agent.
- `utils/logger.py`
  - **What:** One function, `get_logger(name)`. It gets the logger for that name and, only the first time (`if not logger.handlers:`), adds a console handler with a timestamp format and the level from `LOG_LEVEL`.
  - **Why:** Every file logs in the same format with one line of code. Without the `if not logger.handlers:` guard, every log line would print twice.
- `utils/prompts.py`
  - **What:** One constant, `PROMPTS_DIR` (`src/prompts`), and one function, `load_prompt(name)`, which opens `PROMPTS_DIR + "/" + name + ".txt"` and returns its text.
  - **Why:** Prompts live in text files, so you can change how an agent behaves without editing Python — and no agent has to open files itself.
- `__init__.py` (every code folder)
  - **What:** An empty file. You create one in every folder that holds Python code — nothing goes inside it.
  - **Why:** Python only treats a folder as a package when this file is there. Without it, an import like `from src.<folder> import ...` fails with `ModuleNotFoundError`.
- `config.py`
  - **What:** A `Config` dataclass with two fields (`openai_api_key`, `log_level`), a helper `require_env(key)` that raises `MissingConfigError` when a setting is empty, and `load_config()`, which calls `load_dotenv()`, reads both settings and returns a `Config`.
  - **Why:** Every file gets its settings from one place. If the key is missing, the app stops at startup with one clear line — without this file, the missing key would show up later as a confusing API error.
- `exceptions.py`
  - **What:** A few small classes that inherit from `Exception` — at least `MissingConfigError`. Each one is just `class Name(Exception): pass`, with a docstring saying when it's raised.

    - **Why:** Code can catch one specific problem (`except MissingConfigError:`). Without named errors you'd catch every `Exception` , and real bugs would be hidden along with the expected problem.
- `graph.py`
    - **What:** One function, `build_graph()` : add the five nodes, add one edge `START → "supervisor"` , and `compile()` .
    - **Why:** Every other move is a `Command` returned by a node, so the Supervisor decides the route while the program runs.
- `schemas.py`
    - **What:** One Pydantic class, `ReviewVerdict` , with `score: int = Field(ge=1, le=10)` and `feedback: str` .
    - **Why:** The Reviewer's score is a checked number the code compares with the pass mark — not a word to search for.
- `state.py`
    - **What:** A `TeamState` TypedDict (`topic` , `research_notes` , `research_done` , `findings` , `draft` , `feedback_history` , `accepted` , `revision_count` , `routing_log`) with a comment naming each field's single writer, and `start_state(topic)` .
    - **Why:** Each field has exactly one writer, so no agent can overwrite another agent's work.
- `tests/integration/test_live_agents.py`
    - **What:** Test functions with the real API, each printing `OK` or `WRONG` : the Writer keeps a source tag; `run_reviewer()` fails a weak draft; a known topic is researched and its sources kept; an unknown topic gives `""` ; Analysis keeps every fact; a known topic goes through all four specialists; an unknown topic makes the team stop.
    - **Why:** It proves the team works from start to finish — and that the Reviewer really rejects bad work.
- `tests/unit/test_config.py`
    - **What:** Three test functions, no API call, each printing `OK` or `WRONG` : `require_env()` raises `MissingConfigError` for a missing and for an empty setting (a made-up setting name, so your `.env` is never touched), and `load_config()` returns a `Config` when a key is set.
    - **Why:** It proves a missing key stops the app with one clear error, with no API call — the first thing that must work.
- `tests/unit/test_lookup.py`
    - **What:** Two test functions, each printing `OK` or `WRONG` : a known topic returns its 5 facts, and an unknown topic returns `""` .
    - **Why:** A wrong topic match shows up here, with no model and no cost.
- `tests/unit/test_supervisor.py`
    - **What:** A `check(label, state, expected)` helper and test functions that build states by hand with `start_state()` , change a few fields, and check what `decide_next()` returns — every step in order and every way to stop. Each prints `OK` or `WRONG` .

    - **Why:** Routing mistakes are found without running the team or paying for model calls.
- `main.py`
    - **What:** `main()` loads the config, builds the graph, reads a topic, runs `graph.invoke(start_state(topic))`, and passes the result to `print_report(result)`. `print_report()` (written in Step 2, extended in Step 5) prints the routing log and then the article, the "couldn't agree" report, or "nothing is known".
    - **Why:** One starting point that reads the topic and prints the result. The team and its routing live in other files.

How imports work in this layout

- Every folder with Python files gets an empty `__init__.py`. It marks the folder as a package, so `src.agents.writer_agent` is importable. (`prompts/` holds only `.txt` files and `data/` only the fact file, so they don't need one.)
- Python looks for imports in the folder you **run from**. `main.py` and `src/` sit side by side in the repo folder, so `python main.py` finds `from src.graph import build_graph` right there.
- Tests live in sub-folders, so run them as modules from the repo folder: `python -m tests.unit.test_supervisor`. The `-m` tells Python to start from the folder you're in — the repo folder — so `src` is found there too.
- Always run commands from the **repo folder**. `src/prompts/` and `data/sample/facts.json` are both found from there.

Left out on purpose (and when they come)

Folder / file in a big template	Why not here	Where it appears
`api/`, `routes.py`	A terminal app, no web server	Project 5 (Doc12)
`db/`, stored runs	Nothing is saved between runs yet	Project 5 (Doc12)
`evaluation/`, LLM-as-judge	The Reviewer is part of the team, not an evaluation suite	Project 5 (Doc13)
`rag/`, a vector store	Research reads a small fact file	Project 3's pattern, if you need it
A checkpointer	No human pause in this flow	Project 3's pattern, if you add one
`Dockerfile`, CI	Nothing to deploy yet	Project 5
pytest, `conftest.py`	Plain test scripts, same as Projects 1-3	Doc13

How the structure grows, step by step

Step	New files	Changed files
Setup	root files, `src/__init__.py`	—
1	shared files, `models/llm.py`, `agents/writer_agent.py`, `prompts/writer.txt`, `main.py`, `tests/unit/test_config.py`, `tests/integration/test_live_agents.py`	—
2	`schemas.py`, `agents/reviewer_agent.py`, `prompts/reviewer.txt`	`writer_agent.py`, `writer.txt`, `main.py`, `test_live_agents.py`
3	`tools/lookup.py`, `data/sample/facts.json`, `agents/research_agent.py`, `prompts/research.txt`, `tests/unit/test_lookup.py`	`main.py`, `test_live_agents.py`
4	`agents/analysis_agent.py`, `prompts/analysis.txt`	`main.py`, `test_live_agents.py`
5	`state.py`, `graph.py`, `agents/supervisor.py`, `tests/unit/test_supervisor.py`	every agent file (+ a node wrapper), `main.py`, `test_live_agents.py`

Setup (do this once, before Step 1)

Read first: Doc01 — Virtual environments, Doc01 — `.env` files. The same steps as Projects 1-3.

1. Go into the repo folder, start git, and make a venv. From the curriculum's root folder (`multi-agent-ai-engineering-curriculum/`):

```
cd project_4_contentforge_multi_agent
# the \ joins the two lines into one folder name
cd ContentForge-Research-Write-Review-Team-Turns-Any-Topic-\
Into-A-Publish-Ready-Article
git init
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

2. Create `.gitignore` **before** your first commit:

```
# secrets — never pushed
.env
# the virtual environment — rebuilt from requirements.txt
.venv/
```

```
# Python's compiled cache files
__pycache__/
*.pyc
```

3. Create `.env` and `.env.example`, the same as before:

```
# .env — never commit this file
OPENAI_API_KEY=sk-your-real-key-here
LOG_LEVEL=INFO
```

```
# .env.example — commit this one
# Copy this file to .env and fill in the real values. Never commit .env.
OPENAI_API_KEY=
LOG_LEVEL=INFO
```

4. Create `requirements.txt`:

```
langchain-core>=1.0
langchain-openai>=1.0
langgraph>=1.0
pydantic>=2.0
python-dotenv>=1.0
```

5. Create an empty `src/__init__.py`, then install the libraries:

```
pip install -r requirements.txt
```

Check it worked: from the repo folder, `python -c "import src; print('ok')"` prints `ok`. It works because the `src/` folder sits right there — that's why every command runs from the repo folder.

6. Create `LICENSE` — the MIT license, so others may reuse your code. Put the current year and your own name on the Copyright line:

```
MIT License

Copyright (c) 2026 Your Name

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
```

```
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

Why This Project Is Good for Your Portfolio

- **What problem it solves:** one complex task split across specialist agents with clear, separate jobs — and a written reason for every routing decision.
- **Why it matters:** it's what "multi-agent system" means on a resume, and what interviews ask about: "walk me through an agent system you built, and why you designed it that way".
- **When you'd build something like this at a job:** content pipelines, multi-step analysis tools, any workflow with separate phases that each deserve their own instructions.
- **How it's built:** a Supervisor routing with `Command`, one writer per state field, a hard-limited revision loop, and routing rules unit-tested on hand-made states.

Problems you'll likely run into, and how to fix them:

Problem	Fix
The Supervisor routes to the wrong agent	The rules are a plain function (`decide_next`) — test it on hand-made states and fix the order of the rules
Writer and Reviewer never agree	`MAX_REVISIONS` ends the loop with a "couldn't agree" report and the full feedback history
The Reviewer approves everything	Written criteria in its prompt, the pass mark in code, and a test that a weak draft fails
Facts get lost between Research and Writer	Tell Analysis to keep every fact, and count the `[source]` tags before and after
One agent overwrites another's work	One writer per state field — each agent updates only its own fields
An unknown topic still becomes an article	Research returns `""` when nothing is found, and the Supervisor stops the run

Problem	Fix
`GraphRecursionError`	Every node must return a `Command`; check a routing rule isn't sending the run in a circle

Built During These Documents

11_multi_agent_systems (on top of Docs 01-10)

Plan Before You Code

See 15_five_projects_index: write down Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks — and draw the team: who reads and who writes each state field.

How To Build This — Step by Step

The project grows step by step, and **after every step it runs**: Step 1 writes articles from made-up findings, Step 2 revises them with a Reviewer, Step 3 starts from a real topic, Step 4 organizes the research, Step 5 hands the whole flow to a Supervisor. Build it yourself first; use the hints if you're stuck, and the solution only to compare.

Every command runs from the **repo root** with the venv active.

How testing works in this project

Story: ContentForge's promise is "it writes only from looked-up facts, and a Reviewer sends weak drafts back until they pass or the limit is hit" — promises a casual try can't confirm. So every step adds the tests for the part it adds: the Writer's grounding, the Reviewer's rejection, the lookup tool, the facts-kept check, the routing rules, the whole pipeline. When something breaks, the newest `WRONG` points straight at the newest code. **If not:** a prompt change could quietly start inventing facts, or approving everything, and you'd only find out from a reader.

- **Two kinds of test:**
  - `tests/unit/` — code that needs no model: the settings, the lookup tool, every routing rule. No API key, no cost, under a second. **When:** after every change to the file it tests — run them all the time.
  - `tests/integration/test_live_agents.py` — the agents and the whole team against the real API, one case per promise. Real API calls (a fraction of a cent). **When:** at the end of every step, and after changing a prompt, `PASS_SCORE` or `MAX_REVISIONS`.
- **One pattern for every case** — so the output never needs judging:

```
def test_something() -> None:
    result = "OK"
    if <it came out wrong>:
        result = "WRONG"
    print(f"case 1 (what it checks) -> {result}")
```

- **An error you expect** (like `MissingConfigError`): start with `result = "WRONG"`, and set `result = "OK"` inside `except TheError:`.
- **One function per case**, all called at the bottom under `if __name__ == "__main__":`. Each file numbers its own cases, and case numbers never change once written.
- **Model answers vary**, so check one fact — a `[source]` tag is kept, a score is below the pass mark, the next step is `research_agent` — never the exact wording.
- **Run from the repo root with `-m`:** `python -m tests.unit.test_supervisor` (dots, no `.py`). Running the file by its path fails with `No module named 'src'`, because Python then starts looking for imports in `tests/unit/`.
- **Got a `WRONG`?** Read that one test function, then the code it calls. Fix the code — change the test only if the test itself checks the wrong thing.

Step 1 — One Specialist's Drafting Logic, Working Alone, No Agent Behavior

Project: ContentForge-Research-Write-Review-Team-Turns-Any-Topic-Into-A-Publish-Ready-Article — Step 1 of 5

- **Where you are now:** Setup is done — nothing runs yet.
- **What's new in this step:** the shared files, `models/llm.py`, and the Writer: `prompts/writer.txt` + `agents/writer_agent.py` (one LCEL chain), called from `main.py` on 3 test inputs. Tests: `test_config.py` and `test_live_agents.py` with case 1.
- **Why:** a team is only as good as its members. Prove the Writer's core skill alone, so no later failure can be blamed on it.
- **How:** `ChatPromptTemplate.from_template(load_prompt("writer")) | get_model() | StrOutputParser()`, wrapped in `run_writer(findings)`.
- **Working after this step:** `python main.py` prints three articles, one from a nearly empty input, and the test files print `OK`.

Read first: Doc05 — LCEL basics.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Bring in the shared files (config, errors, logger, prompt loader with folder `src/prompts`) and `models/llm.py`.

2. Write `prompts/writer.txt` : a titled 150-250-word article, **only** from the findings, keeping each `[source]` tag.
3. Write `agents/writer_agent.py` : `build_writer_chain()` and `run_writer(findings)` .
4. In `main.py` , run the Writer on 3 made-up findings — one of them almost empty — and **read** the results.
5. Write the first tests — `tests/unit/test_config.py` and `tests/integration/test_live_agents.py` with case 1 — and run them with `python -m` .

The root files — `.gitignore` , `.env` , `.env.example` , `requirements.txt` and `LICENSE` — come from Setup; this step adds the files below.

Your files after Step 1:

```
<repo folder>/
├── src/
│   ├── agents/
│   │   ├── __init__.py      (new)
│   │   └── writer_agent.py  (new) build_writer_chain(), run_writer()
│   ├── models/
│   │   ├── __init__.py      (new)
│   │   └── llm.py           (new) get_model()
│   ├── prompts/
│   │   └── writer.txt       (new)
│   ├── utils/
│   │   ├── __init__.py      (new)
│   │   ├── logger.py        (new)
│   │   └── prompts.py       (new)
│   ├── __init__.py
│   ├── config.py            (new)
│   └── exceptions.py        (new)
├── tests/
│   ├── integration/
│   │   └── test_live_agents.py (new) case 1
│   └── unit/
│       └── test_config.py   (new) no API key needed
└── main.py                  (new) runs the Writer on 3 test inputs
```

Why these files (what and why):

- `agents/writer_agent.py`
  - **What:** `build_writer_chain()` and `run_writer(findings)` — one LCEL chain turning findings into an article.
  - **Why:** It writes the article; wrapping the chain in `run_writer()` means Step 2 can add feedback without touching `main.py` .
- `models/llm.py`

    - **What:** A `MODEL_NAME` constant and one function, `get_model()`, which returns `ChatOpenAI(model=MODEL_NAME, temperature=0)`.
    - **Why:** Every chain and agent gets the same model and settings from here. To try another model you change one line, not every file.
- `prompts/writer.txt`
    - **What:** This text file holds the writer prompt. The code loads it with `load_prompt("writer")`.
    - **Why:** Each agent's narrow job is written down in its own prompt — the reason for splitting the work in the first place.
- `utils/logger.py`
    - **What:** One function, `get_logger(name)`. It gets the logger for that name and, only the first time (`if not logger.handlers:`), adds a console handler with a timestamp format and the level from `LOG_LEVEL`.
    - **Why:** Every file logs in the same format with one line of code. Without the `if not logger.handlers:` guard, every log line would print twice.
- `utils/prompts.py`
    - **What:** One constant, `PROMPTS_DIR` (`src/prompts`), and one function, `load_prompt(name)`, which opens `PROMPTS_DIR + "/" + name + ".txt"` and returns its text.
    - **Why:** Prompts live in text files, so you can change how an agent behaves without editing Python — and no agent has to open files itself.
- `config.py`
    - **What:** A `Config` dataclass with two fields (`openai_api_key`, `log_level`), a helper `require_env(key)` that raises `MissingConfigError` when a setting is empty, and `load_config()`, which calls `load_dotenv()`, reads both settings and returns a `Config`.
    - **Why:** Every file gets its settings from one place. If the key is missing, the app stops at startup with one clear line — without this file, the missing key would show up later as a confusing API error.
- `exceptions.py`
    - **What:** A few small classes that inherit from `Exception` — at least `MissingConfigError`. Each one is just `class Name(Exception): pass`, with a docstring saying when it's raised.
    - **Why:** Code can catch one specific problem (`except MissingConfigError:`). Without named errors you'd catch every `Exception`, and real bugs would be hidden along with the expected problem.
- `test_config.py`
    - **What:** Three test functions, no API call, each printing `OK` or `WRONG`: `require_env()` raises `MissingConfigError` for a missing and for an empty setting (a made-up setting name, so your `.env` is never touched), and `load_config()` returns a `Config` when a key is set.

    - **Why:** It proves a missing key stops the app with one clear error, with no API call — the first thing that must work.
- `test_live_agents.py`
    - **What:** One test function against the real API (case 1): the Writer turns findings into an article that keeps its `[source]` tag.
    - **Why:** It proves the key, the model, the prompt file and the chain work together.
- `main.py`
    - **What:** `main()` loads the config and runs `run_writer()` on three made-up findings, printing each article. Later steps replace this with the full team.
    - **Why:** One starting point that proves the Writer alone before any other agent exists.

Tests for this step — write them yourself (the solution has the code, to compare):

- **case 1 (missing setting)** — `tests/unit/test_config.py`
    - **Check:** with a made-up setting name removed from `os.environ`, `require_env()` raises `MissingConfigError`.
    - **Why:** a missing key must stop the app at startup with one clear line.
    - **When:** after any change to `config.py` or `exceptions.py`.
    - **If not:** a missing key would surface later as a confusing `AuthenticationError` from OpenAI.
- **case 2 (empty setting)** — `tests/unit/test_config.py`
    - **Check:** the same name set to `""` also raises `MissingConfigError`.
    - **Why:** `OPENAI_API_KEY=` with nothing after it is "set" but never usable.
    - **When:** same as case 1.
    - **If not:** the app would start happily and fail on the first real call.
- **case 3 (config loads)** — `tests/unit/test_config.py`
    - **Check:** with `OPENAI_API_KEY` set in `os.environ`, `load_config()` returns a `Config` holding that key.
    - **Why:** cases 1–2 only prove the *error* path; this proves the values actually come back.
    - **When:** same as case 1.
    - **If not:** a missing `return value` in `require_env()` passes cases 1–2, and every setting silently becomes `None`.
- **case 1 (writer keeps sources)** — `tests/integration/test_live_agents.py`
    - **Check:** `run_writer()` on findings with a `[source]` tag returns an article that still contains a `[` tag.
    - **Why:** the Writer's one rule is "only the findings, keep the tags" — this proves it followed it.
    - **When:** at the end of the step, and after changing `writer.txt` or `writer_agent.py`.
    - **If not:** a Writer that drops sources would only be caught by a reader checking every fact by hand.

Step 2 — That Specialist, Now a Real Agent That Can Improve Its Own Work

Project: ContentForge-Research-Write-Review-Team-Turns-Any-Topic-Into-A-Publish-Ready-Article — Step 2 of 5

- **Where you are now:** after Step 1 the Writer drafts once — it can't take criticism.
- **What's new in this step:** a `feedback` parameter for the Writer, a Reviewer (`schemas.py` + `prompts/reviewer.txt` + `agents/reviewer_agent.py`) that scores drafts, and a limited write → review → revise loop in `main.py` . Tests: case 2 in `test_live_agents.py` .
- **What stays the same:** the Writer's chain itself.
- **Why:** an agent that can't improve under criticism isn't really agentic; proving the loop alone means Step 5 can trust it.
- **How:** `run_reviewer()` returns a checked `ReviewVerdict(score, feedback)` ; the code compares the score to `PASS_SCORE` ; `write_with_review()` feeds the feedback back, up to `MAX_REVISIONS` rounds, and returns a dict (`draft` , `accepted` , `feedback_history` , `revision_count`) that `print_report(result)` prints.
- **Working after this step:** a draft is written, scored, revised, and either accepted or reported as "couldn't agree".

Read first: Doc11 — patterns (generator → critic), Doc05 — structured parser, Doc11 — convergence exercise.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Add a feedback section to `writer.txt` , and a `feedback` parameter to `run_writer()` .
2. Write `ReviewVerdict` (`score: int` 1-10, `feedback: str`) in `schemas.py` .
3. Write `prompts/reviewer.txt` with three **written** criteria (clarity, grounding, completeness), and `agents/reviewer_agent.py` with `PASS_SCORE = 7` and `run_reviewer()` .
4. In `main.py` , write `write_with_review(findings)` : up to `MAX_REVISIONS` rounds, returning a dict with `draft` , `accepted` , `feedback_history` and `revision_count` — the same keys Step 5's team state will use. Then write `print_report(result)` , which never shows a failed draft as finished. Both keep their names for the rest of the project.
5. Add case 2 to the integration test: a weak draft **must** fail.

Your files after Step 2:

```
<repo folder>/
├── src/
│   ├── agents/
│   │   ├── reviewer_agent.py (new) PASS_SCORE, run_reviewer()
│   │   └── writer_agent.py  (changed) + feedback
│   ├── prompts/
```

```
│   │   ├── reviewer.txt     (new)
│   │   └── writer.txt       (changed) + feedback section
│   └── schemas.py           (new) ReviewVerdict
├── tests/integration/test_live_agents.py (changed) + case 2
└── main.py                  (changed) write_with_review(), print_report()
```

Why these files (what and why):

- `agents/reviewer_agent.py`
  - **What:** `PASS_SCORE`, and `run_reviewer(findings, draft)` — a chain with `with_structured_output(ReviewVerdict)` that scores the draft.
  - **Why:** A second opinion with written criteria stops weak drafts; the pass mark lives in code, not the model's mood.
- `agents/writer_agent.py`
  - **What:** `build_writer_chain()` and `run_writer(findings, feedback="")` — the same chain, now able to be told what to fix.
  - **Why:** Feedback is what turns a one-shot chain into an agent that can improve its own work.
- `prompts/reviewer.txt`
  - **What:** This text file holds the reviewer prompt. The code loads it with `load_prompt("reviewer")`.
  - **Why:** The Reviewer's three criteria are the most-edited text in the project; they shouldn't be buried in code.
- `prompts/writer.txt`
  - **What:** The writer prompt, now with a feedback section the Writer must address.
  - **Why:** Each agent's narrow job is written down in its own prompt.
- `schemas.py`
  - **What:** One Pydantic class, `ReviewVerdict`, with `score: int = Field(ge=1, le=10)` and `feedback: str`.
  - **Why:** The Reviewer's score is a checked number the code compares with the pass mark — not a word to search for.
- `main.py`
  - **What:** `write_with_review(findings)` loops the Writer and Reviewer up to `MAX_REVISIONS`, returning a dict; `print_report(result)` prints it, never passing a rejected draft off as finished.
  - **Why:** The revision loop is proven here, in plain code, before Step 5 moves it into the graph — and the result shape is the one the team state will reuse.
- `test_live_agents.py`
  - **What:** Adds case 2 — `run_reviewer()` must fail a weak draft.
  - **Why:** A Reviewer that passes everything looks exactly like one that works.

Tests for this step — write them yourself:

- **case 2 (weak draft fails)** — `tests/integration/test_live_agents.py`
  - **Check:** `run_reviewer(findings, "E-bikes are good.")` returns a score below `PASS_SCORE`.
  - **Why:** the whole revision loop only ever runs if the Reviewer actually rejects weak work.
  - **When:** at the end of the step, and after changing `reviewer.txt` or `PASS_SCORE`.
  - **If not:** a Reviewer that approves everything would make the loop pointless, and you'd never know.

Step 3 — A Research Agent That Looks Things Up Before Anyone Else Works

Project: ContentForge-Research-Write-Review-Team-Turns-Any-Topic-Into-A-Publish-Ready-Article — Step 3 of 5

- **Where you are now:** after Step 2 the Writer and Reviewer work together — but on made-up findings.
- **What's new in this step:** a lookup tool (`tools/lookup.py` + `data/sample/facts.json`) and the Research agent, which looks facts up **first**, then turns them into notes; `main.py` now starts from a topic. Tests: `test_lookup.py`, and cases 3-4 in `test_live_agents.py`.
- **What stays the same:** the Writer, the Reviewer, `write_with_review()` and `print_report()`.
- **Why:** everything downstream is only as true as the research it starts from.
- **How:** `lookup_facts.invoke({"topic": ...})` → nothing found? return `""` → otherwise an LCEL chain writes one note per fact, keeping the `[source]` tags.
- **Working after this step:** type a topic; get research notes and a reviewed article — or an honest "nothing is known" for an unknown topic.

Read first: Doc06 — first tool, Doc02 — JSON.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write `data/sample/facts.json` (a few topics, 4-5 facts each, each with a `[source]` tag) and `tools/lookup.py` with the `lookup_facts` tool. Test the tool **alone** first (`tests/unit/test_lookup.py`).
2. Write `prompts/research.txt` and `agents/research_agent.py` with `run_research(topic)`.
3. Change `main.py`: ask for a topic, run Research, stop honestly if it's empty, then write and review from the notes.
4. Add research cases to the integration test: a known topic keeps its sources; an unknown one gives `""`.

Your files after Step 3:

```
<repo folder>/
├── data/
│   └── sample/
│       └── facts.json       (new)
├── src/
│   ├── agents/
│   │   └── research_agent.py (new) run_research()
│   ├── prompts/
│   │   └── research.txt     (new)
│   └── tools/
│       ├── __init__.py      (new)
│       └── lookup.py        (new) lookup_facts
├── tests/
│   ├── integration/test_live_agents.py (changed) + cases 3-4
│   └── unit/test_lookup.py  (new)
└── main.py                  (changed) topic -> research -> write
```

Why these files (what and why):

- `data/sample/facts.json`
  - **What:** A JSON object mapping each topic to a list of short facts, each ending with a `[source]` tag.
  - **Why:** You know these facts, so an invented fact in the article is easy to spot.
- `agents/research_agent.py`
  - **What:** `run_research(topic)` calls the lookup tool first; if nothing is found it returns `""`, otherwise an LCEL chain turns the facts into notes.
  - **Why:** The article starts from looked-up facts, and an unknown topic stops the run instead of inventing facts.
- `prompts/research.txt`
  - **What:** This text file holds the research prompt. The code loads it with `load_prompt("research")`.
  - **Why:** Each agent's narrow job is written down in its own prompt.
- `tools/lookup.py`
  - **What:** `load_facts()` reads `facts.json` with `json.loads()`, and a `@tool` function `lookup_facts(topic)` returns the facts of the first known topic found in the text, one per line — or `""`.
  - **Why:** Research's tool can be tested without any model, so a wrong note can be pinned on the tool or on the agent.
- `test_lookup.py`
  - **What:** Two test functions, each printing `OK` or `WRONG`: a known topic returns its 5 facts, and an unknown topic returns `""`.
  - **Why:** A wrong topic match shows up here, with no model and no cost.

- `test_live_agents.py`
  - **What:** Adds case 3 (a known topic keeps its sources) and case 4 (an unknown topic gives `""`).
  - **Why:** It proves Research grounds its notes in real facts and never invents them.
- `main.py`
  - **What:** Now asks for a topic, runs Research, stops honestly when nothing is found, and hands the notes to the same `write_with_review()` loop.
  - **Why:** The app now starts from a real topic, not a hard-coded sample.

Tests for this step — write them yourself:

- **case 1 (known topic -> 5 facts)** — `tests/unit/test_lookup.py`
  - **Check:** `lookup_facts.invoke({"topic": "Why Electric Bikes are popular"})` returns 5 lines.
  - **Why:** the agent's notes can only be as good as the facts the tool hands it.
  - **When:** after any change to `lookup.py` or `facts.json`.
  - **If not:** a wrong substring match would quietly hand the wrong topic's facts to the Writer.
- **case 2 (unknown topic -> empty)** — `tests/unit/test_lookup.py`
  - **Check:** `lookup_facts.invoke({"topic": "deep sea mining"})` returns `""`.
  - **Why:** "nothing known" must be an empty string the agent can act on, not a guess.
  - **When:** same as case 1.
  - **If not:** a tool that returned something for an unknown topic would let the team write an article about nothing.
- **case 3 (known topic sourced)** — `tests/integration/test_live_agents.py`
  - **Check:** `run_research("electric bikes")` returns notes with at least 5 `[source]` tags.
  - **Why:** the research prompt must keep every tag, not summarize them away.
  - **When:** at the end of the step, and after changing `research.txt`.
  - **If not:** notes without tags would make the Writer's sourced article impossible.
- **case 4 (unknown topic empty)** — `tests/integration/test_live_agents.py`
  - **Check:** `run_research("deep sea mining")` returns `""` — no model call, no invented notes.
  - **Why:** the agent must stop at an unknown topic, not ask the model to make something up.
  - **When:** same as case 3.
  - **If not:** an unknown topic would still produce a confident, invented article.

Step 4 — An Analysis Agent That Turns Raw Research Into Organized Findings

Project: ContentForge-Research-Write-Review-Team-Turns-Any-Topic-Into-A-Publish-Ready-Article — Step 4 of 5

- **Where you are now:** after Step 3 the Writer drafts straight from a flat list of notes.

- **What's new in this step:** the Analysis agent (`agents/analysis_agent.py` + `prompts/analysis.txt`), which groups the notes into themes without losing a fact; `main.py` runs it between Research and the writing loop. Tests: case 5 in `test_live_agents.py` .
- **What stays the same:** Research, Writer, Reviewer.
- **Why:** the first time one agent's **real** output feeds another — an interface mismatch shows up now, before a Supervisor hides it.
- **How:** one LCEL chain; the test counts `[source]` tags in Research's real output and in Analysis's result.
- **Working after this step:** topic → research → analysis → reviewed article.

Read first: Doc05 — LCEL basics.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write `prompts/analysis.txt` : 2-4 themes, **keep every fact and its tag**, and an "Other" theme for facts that don't fit.
2. Write `agents/analysis_agent.py` with `run_analysis(notes)` .
3. Change `main.py` to run Analysis before the writing loop.
4. Add a test that feeds Research's **real** output to Analysis and compares the tag counts.

Your files after Step 4:

```
<repo folder>/
├── src/
│   ├── agents/
│   │   └── analysis_agent.py (new) run_analysis()
│   └── prompts/
│       └── analysis.txt     (new)
├── tests/integration/test_live_agents.py (changed) + case 5
└── main.py                  (changed) research -> analysis -> write
```

Why these files (what and why):

- `agents/analysis_agent.py`
  - **What:** `run_analysis(notes)` runs a plain LCEL chain that groups the notes into themes while keeping every fact.
  - **Why:** The Writer gets a clear structure, and the "keep every fact" rule means nothing is lost on the way.
- `prompts/analysis.txt`
  - **What:** This text file holds the analysis prompt. The code loads it with `load_prompt("analysis")` .

    - **Why:** Each agent's narrow job is written down in its own prompt.
- `test_live_agents.py`
    - **What:** Adds case 5 — Research's real output goes in, and the `[source]` tags are counted before and after.
    - **Why:** It proves Analysis organizes without quietly dropping a fact.
- `main.py`
    - **What:** Now runs Analysis between Research and the writing loop.
    - **Why:** The Writer drafts from themed findings, not a flat list of notes.

Tests for this step — write them yourself:

- **case 5 (analysis keeps facts)** — `tests/integration/test_live_agents.py`
    - **Check:** feed `run_research("electric bikes")` into `run_analysis()`; the `[source]` tag count is the same in and out.
    - **Why:** organizing must not silently lose a fact that didn't fit a theme.
    - **When:** at the end of the step, and after changing `analysis.txt`.
    - **If not:** a dropped fact would make the article quietly incomplete, with nothing flagging it.

Step 5 — The Full Team: a Supervisor Routing 4 Specialists via Command (final: 5 agents)

Project: ContentForge-Research-Write-Review-Team-Turns-Any-Topic-Into-A-Publish-Ready-Article — Step 5 of 5

- **Where you are now:** after Step 4 all four specialists work — but `main.py` runs them in a fixed order with a hand-written loop.
- **What's new in this step:** `state.py` (the shared state, with the same keys `print_report()` already reads), `agents/supervisor.py` (routing rules + `Command`), `graph.py`, a node wrapper in every agent file, and the Supervisor's unit test. `print_report()` gains the routing log and a "nothing is known" branch; `write_with_review()` is removed — the Supervisor and `reviewer_node` now do its job.
- **What stays the same:** every agent's `run_...()` function, every prompt, the tool.
- **Why:** this is the actual multi-agent system: separately proven specialists, coordinated by a Supervisor that logs why it routed each way.
- **How:** each node returns `Command(update=..., goto="supervisor")`; `decide_next(state)` picks the next step and a reason; the Reviewer clears a rejected draft to send it back to the Writer; `MAX_REVISIONS` stops the loop.
- **Working after this step:** the full project — a topic becomes a reviewed article, with a routing log explaining every move.

Read first: Doc11 — the Command tool, Doc11 — shared state, Doc11 Build Task.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write `state.py`: `TeamState` with **one writer per field**, plus `start_state(topic)`.
2. Write `agents/supervisor.py`: `MAX_REVISIONS`, `decide_next(state) -> (step, reason)`, and `supervisor(state)`, which logs the decision and returns a `Command`. Test `decide_next` on hand-made states **before** wiring the graph.
3. Add a node wrapper to each agent file (`research_node`, `analysis_node`, `writer_node`, `reviewer_node`); the Reviewer's node holds Step 2's loop logic.
4. Write `graph.py`: five nodes, one edge (`START → supervisor`).
5. Change `main()` to run the graph and pass the result to the same `print_report()`; add the routing log and the "nothing is known" branch to `print_report()`; delete `write_with_review()`.
6. Update the integration test: keep cases 1-5, add case 6 (a full pipeline) and case 7 (an unknown topic stops).

Your files after Step 5 (final): the full tree is in Project Structure. What changed:

```
<repo folder>/
├── src/
│   ├── agents/
│   │   ├── analysis_agent.py  (changed) + analysis_node
│   │   ├── research_agent.py  (changed) + research_node
│   │   ├── reviewer_agent.py  (changed) + reviewer_node
│   │   ├── supervisor.py      (new) decide_next(), supervisor()
│   │   └── writer_agent.py    (changed) + writer_node
│   ├── graph.py               (new) 5 nodes, 1 edge
│   └── state.py               (new) TeamState — one writer per field
├── tests/
│   ├── integration/test_live_agents.py (changed) + cases 6-7
│   └── unit/test_supervisor.py         (new)
└── main.py                    (changed) runs the graph; print_report() +2
```

Why these files (what and why):

- `agents/analysis_agent.py`
  - **What:** Keeps `run_analysis()`; adds `analysis_node(state)`, which returns `Command(update={findings}, goto="supervisor")`.
  - **Why:** The same skill, now callable as a graph node.
- `agents/research_agent.py`
  - **What:** Keeps `run_research()`; adds `research_node(state)`, which sets `research_done` even when nothing is found.
  - **Why:** `research_done` is what lets the Supervisor avoid running the same lookup twice.
- `agents/reviewer_agent.py`

    - **What:** Keeps `run_reviewer()`; adds `reviewer_node(state)` holding Step 2's loop logic — set `accepted` on a pass, else record the feedback and clear the draft while revisions remain.
    - **Why:** Clearing the draft is the signal that sends the work back to the Writer.
- `agents/supervisor.py`
    - **What:** `MAX_REVISIONS`, `decide_next(state)` returning `(step, reason)`, and `supervisor(state)`, which logs the decision and returns `Command(goto=...)`.
    - **Why:** The routing rules are a plain function you can test with a hand-made dict, and every move is logged with its reason.
- `agents/writer_agent.py`
    - **What:** Keeps `run_writer()`; adds `writer_node(state)`, which passes the newest feedback from `feedback_history`.
    - **Why:** The Writer revises with the Reviewer's exact words, read from the shared state.
- `graph.py`
    - **What:** One function, `build_graph()`: add the five nodes, one edge `START → "supervisor"`, and `compile()`.
    - **Why:** Every other move is a `Command` returned by a node, so the Supervisor decides the route while the program runs.
- `state.py`
    - **What:** A `TeamState` TypedDict with one writer per field, and `start_state(topic)`.
    - **Why:** Each field has exactly one writer, so no agent can overwrite another agent's work.
- `test_supervisor.py`
    - **What:** A `check(label, state, expected)` helper and test functions that build states by hand and check every routing decision — every step in order and every way to stop. Each prints `OK` or `WRONG`.
    - **Why:** Routing mistakes are found without running the team or paying for model calls.
- `test_live_agents.py`
    - **What:** Keeps cases 1-5; adds case 6 (a known topic goes through all four specialists) and case 7 (an unknown topic makes the team stop).
    - **Why:** It proves the whole team works end to end, not just each agent alone.
- `main.py`
    - **What:** `main()` now builds the graph and runs it; `print_report()` gains the routing log and a "nothing is known" branch; `write_with_review()` is removed.
    - **Why:** The order of work now lives in the Supervisor, with a shared state to inspect and a routing log to explain every run.

Tests for this step — write them yourself:

- **cases 1-8 (routing rules)** — `tests/unit/test_supervisor.py`

    - **Check:** `decide_next()` returns the right next step for a fresh start, research done, findings ready, a draft ready, nothing found, a rejected draft, the limit reached, and an accepted draft.
    - **Why:** the order of the `if` checks *is* the routing policy — one wrong order sends the whole team the wrong way.
    - **When:** after any change to `decide_next()` or `MAX_REVISIONS`.
    - **If not:** a routing bug would only show up as a strange, expensive full run.
- **case 6 (full pipeline)** — `tests/integration/test_live_agents.py`
    - **Check:** a known topic's routing log includes research, analysis, writer and reviewer.
    - **Why:** proves the five nodes, the shared state and the `Command` handoffs all wire together.
    - **When:** at the end of the step, and after changing `graph.py` or any node wrapper.
    - **If not:** a broken handoff would be found only by watching a real run fail.
- **case 7 (unknown topic stops)** — `tests/integration/test_live_agents.py`
    - **Check:** an unknown topic ends at `END` with the reason "research found no facts".
    - **Why:** the team must stop honestly, not invent an article, when research is empty.
    - **When:** same as case 6.
    - **If not:** an unknown topic would loop or produce a confident, baseless article.

Final Deliverable: ContentForge-Research-Write-Review-Team-Turns-Any-Topic-Into-A-Publish-Ready-Article — a Supervisor routing Research, Analysis, Writer and Reviewer with `Command`, over a shared state with one writer per field, with a hard-limited revision loop and a reason logged for every move.

Why "4 agents and not 1" (said plainly): each specialist has one narrow job and its own prompt, so each can be tested and fixed alone; the Reviewer can reject what the Writer produced; and a wrong fact can be traced to the step that introduced it. The cost is more model calls — for a short article, one call is cheaper. Use your Doc11 cost/speed comparison when you defend the choice.

Run, Test and Configure the Final Project

The same commands and settings as the repo's README.md — once Step 5 is done, this is how the finished project is run, tested and configured.

Configuration

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	`DEBUG`, `INFO`, `WARNING` or `ERROR`

Setting	Where	Default
Chat model	`src/models/llm.py` → `MODEL_NAME`	`gpt-4o-mini`
Pass mark	`src/agents/reviewer_agent.py` → `PASS_SCORE`	`7` (out of 10)
Revision limit	`src/agents/supervisor.py` → `MAX_REVISIONS`	`3`
Known facts	`data/sample/facts.json`	electric bikes, remote work, solar panels

To support a new topic, add it with its facts and `[source]` tags to `facts.json`.

Usage

Run from the repo root:

```
python main.py
```

Enter a topic. You get the routing log and either the final article, a "couldn't agree" report with every round's feedback, or "nothing is known about this topic yet".

Running the Tests

```
# unit tests — no API key
python -m tests.unit.test_config
python -m tests.unit.test_lookup
python -m tests.unit.test_supervisor

# integration tests — real API calls, needs OPENAI_API_KEY
python -m tests.integration.test_live_agents
```

The unit tests check the settings, the lookup tool and every routing rule on hand-made states. The integration test checks that the Writer keeps its sources, the Reviewer fails a weak draft, Research grounds and stops honestly, Analysis keeps every fact, and the whole team runs end to end.

Checklist Before You Call This Done

- [ ] The repo folder name matches the GitHub repo name exactly
- [ ] Code lives in `src/`, `main.py` at the repo root; runs with `python main.py`
- [ ] Every folder has one clear job, and you can say why it exists

- [ ] The Supervisor routes with `Command`, and logs a reason for every move
- [ ] The routing rules are tested on hand-made states, with no model
- [ ] One writer per state field — no agent overwrites another's work
- [ ] The Reviewer rejects a weak draft in a real test; `MAX_REVISIONS` ends a loop that never agrees
- [ ] Research never runs twice, and an unknown topic stops the run
- [ ] The full task goes through all four specialists in a real run
- [ ] `tests/unit/` pass with no API key; `tests/integration/` pass with one
- [ ] You can defend "why 4 agents and not 1", with your Doc11 cost/speed numbers

Publish It to GitHub

1. Copy README.md from this folder into your repo folder, and put your own name in its License line.
2. Check nothing secret is staged, then commit:

```
git status                     # .env and .venv/ must NOT appear
git add .
git commit -m "ContentForge: supervisor-led research, write, review team"
```

3. On GitHub, create a new repo named exactly `ContentForge-Research-Write-Review-Team-Turns-Any-Topic-Into-A-Publish-Ready-Article` (no README — you have one). In the repo's **Description** box, paste the **Short description** line from the top of README.md. Then:

```
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

Status

Not started. Track your progress in ../PROGRESS.md.

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**